

Every Concept You'll Use, Explained Simply

A plain-language study guide to the machine-learning ideas behind the telecom churn project — what each one means, a worked example from the project, and exactly how to say it to an interviewer.

How to use this: read one concept at a time. First understand the **plain explanation**, then the **example** to make it concrete, then memorise the **"Say it in the interview"** line so you can explain it out loud in one breath.

Prepared for Lavish · B.Tech CS (AI/ML) · Target: Airtel AI/ML role

Contents

A • THE PROBLEM & SETUP

1. Customer churn (the business problem)
2. Supervised learning & classification
3. Features and the target variable

B • PREPARING THE DATA

1. Train / test split & stratification
2. Data leakage
3. Handling missing values
4. Encoding categorical variables
5. Feature scaling

C • THE IMBALANCE PROBLEM

1. Class imbalance (why accuracy lies)
2. SMOTE & class weights

D • THE MODELS

1. Logistic Regression
2. Decision Tree
3. Random Forest (bagging)
4. Gradient Boosting & XGBoost
5. Bias–variance tradeoff
6. Overfitting, underfitting & regularization (L1/L2)
7. Cross-validation
8. Hyperparameter tuning

E • MEASURING THE MODEL

1. Confusion matrix
2. Precision, Recall & F1
3. ROC-AUC & PR-AUC
4. Threshold tuning

F • EXPLAINING & USING IT (THE DIFFERENTIATOR)

1. Feature importance & SHAP
2. Customer value & risk ranking

3. Uplift modeling & retention ROI

A · The Problem & Setup

1 Customer churn (the business problem)

IN SIMPLE WORDS Churn just means a customer **leaving**. For a telecom company, a churned customer is someone who stops recharging or ports their number to another operator. We want to catch them *before* they go, because keeping a customer is much cheaper than winning a new one.

EXAMPLE A prepaid user whose recharges dropped from monthly to nothing in 60 days, who called support twice about network issues — high churn risk.

SAY IT IN THE INTERVIEW "Churn is when a customer stops using the service. My model predicts churn probability per customer so the business can target retention offers before they leave — since retaining a customer costs far less than acquiring one."

2 Supervised learning & classification

IN SIMPLE WORDS **Supervised learning** = we learn from examples where we already know the answer. Here, past customers are labelled "churned" or "stayed", and the model learns the pattern. Because the answer is a category (Yes/No), this is a **classification** problem (not regression, which predicts a number).

EXAMPLE We feed 7,000 past customers with their known outcome. The model learns "short tenure + monthly contract + high charges → likely churn," then applies it to new customers.

SAY IT IN THE INTERVIEW "It's a supervised binary classification problem — I train on historical customers whose churn outcome is known, and predict Yes/No churn for new ones."

3 Features and the target variable

IN SIMPLE WORDS **Features** (X) are the inputs — the facts about a customer. The **target** (y) is what we predict — here, **Churn**. The model learns a mapping from X to y.

TENURE	MONTHLYCHARGES	CONTRACT	COMPLAINTS	CHURN (TARGET)
2	₹399	Month-to-month	3	Yes
40	₹599	Two-year	0	No

SAY IT IN THE INTERVIEW "Each row is a customer; the feature columns describe their behaviour, and the target column is whether they churned. The model learns the relationship between the two."

B · Preparing the Data

4 Train / test split & stratification

IN SIMPLE WORDS We split data into a **training set** (the model learns from it) and a **test set** (we check it on unseen data). Typically 80/20. **Stratified** split keeps the same churn ratio in both parts, so the test set is a fair mini-version of reality.

EXAMPLE If 26% of all customers churned, a stratified split keeps ~26% churners in both train and test — otherwise the test set might get too few churners and give misleading scores.

SAY IT IN THE INTERVIEW "I use a stratified 80/20 split so both sets keep the real churn ratio — the test set stays a fair check on unseen data."

5 Data leakage

IN SIMPLE WORDS Leakage is when information the model shouldn't have "leaks" from the test set (or from the future) into training. It makes scores look amazing in testing but the model fails in the real world.

EXAMPLE If I calculate the average of a column using the *whole* dataset before splitting, the test set's info leaks into training. Fix: fit scalers/encoders on the **training set only**, then apply to test. Also drop columns that basically reveal the answer (e.g. "cancellation date").

SAY IT IN THE INTERVIEW "Leakage is when test or future information sneaks into training. I prevent it by fitting all preprocessing on the training split only, then transforming the test split with it."

6 Handling missing values

IN SIMPLE WORDS Real data has blanks. We either **fill** them (imputation) or drop them. Numbers are often filled with the median; categories with the most common value or a "Unknown" label.

EXAMPLE In the Telco dataset, `TotalCharges` has a few blank strings (new customers). We convert to number and fill the blanks with the median — dropping them would throw away real customers.

SAY IT IN THE INTERVIEW "I handle missing values with median imputation for numbers and a most-frequent/'Unknown' category for text, deciding per column rather than blindly dropping rows."

7 Encoding categorical variables

IN SIMPLE WORDS Models need numbers, not words. **One-hot encoding** makes a 0/1 column for each category (good when there's no order). **Label encoding** maps categories to integers (only for ordered categories, else the model invents a fake order).

EXAMPLE `Contract` = {Month-to-month, One-year, Two-year} → three 0/1 columns via one-hot. Using label encoding (0,1,2) here would wrongly tell the model "Two-year is twice One-year."

SAY IT IN THE INTERVIEW "I one-hot encode unordered categories so the model doesn't assume a false ranking, and only label-encode when the category is genuinely ordinal."

8 Feature scaling

IN SIMPLE WORDS Putting features on a similar range. `tenure` (0–72) and `MonthlyCharges` (0–1000+) live on very different scales; some models get biased toward the bigger numbers. **Standardization** rescales to mean 0, std 1.

EXAMPLE Logistic Regression and KNN need scaling; tree models (Random Forest, XGBoost) don't really care, because they split on thresholds, not distances.

SAY IT IN THE INTERVIEW "I scale features for distance/gradient-based models like Logistic Regression, but skip it for tree-based models since splits are scale-invariant."

C · The Imbalance Problem

9 Class imbalance (why accuracy lies)

IN SIMPLE WORDS When one class is rare — few churners vs many stayers — a lazy model that predicts "nobody churns" can still score high accuracy while being useless. This is the single most important trap in churn/fraud.

EXAMPLE If only 5% churn, predicting "no churn" for everyone gives **95% accuracy** — but catches **zero** churners. That's why we use recall/AUC instead.

SAY IT IN THE INTERVIEW "Churn data is imbalanced, so accuracy is misleading — a model can hit 95% by predicting nobody leaves. I judge it on recall and ROC-AUC and I rebalance during training."

10 SMOTE & class weights

IN SIMPLE WORDS Two ways to fix imbalance. **Class weights** tell the model "mistakes on the rare class hurt more." **SMOTE** creates synthetic (fake but realistic) churning examples to balance the training data.

EXAMPLE In sklearn, `class_weight='balanced'` is often enough. SMOTE is applied to the **training set only** (never the test set — that would be leakage).

SAY IT IN THE INTERVIEW "I handle imbalance with class weights, or SMOTE to synthesise minority samples — applied to training data only so the test set stays honest."

D · The Models

11 Logistic Regression

IN SIMPLE WORDS A simple, fast model that draws a line/boundary and outputs a **probability** between 0 and 1 using the sigmoid function. Great as a **baseline** and easy to interpret (each feature has a weight).

EXAMPLE It might learn: month-to-month contract pushes churn probability up, long tenure pushes it down. Output: "0.82 → 82% likely to churn."

SAY IT IN THE INTERVIEW "I start with Logistic Regression as an interpretable baseline — it outputs a churn probability via the sigmoid and its weights show each feature's direction."

12 Decision Tree

IN SIMPLE WORDS A flowchart of yes/no questions. It splits data on the feature that best separates churners from stayers, again and again. Easy to read, but a single deep tree **overfits** (memorises the training data).

EXAMPLE "Contract = month-to-month? → yes → tenure < 6 months? → yes → predict churn."

SAY IT IN THE INTERVIEW "A decision tree splits on the most informative features. It's interpretable but overfits alone, which is why we use ensembles of trees."

13 Random Forest (bagging)

IN SIMPLE WORDS Instead of one tree, build **many** trees on random subsets of data and features, then **vote**. Averaging many opinions cancels out individual mistakes — this is **bagging**. More accurate and far less overfitting than one tree.

EXAMPLE 200 trees each guess churn/stay; if 150 say "churn", the forest says churn (probability 0.75).

SAY IT IN THE INTERVIEW "Random Forest is bagging — many de-correlated trees vote, which reduces variance and overfitting compared to a single tree."

14 Gradient Boosting & XGBoost

IN SIMPLE WORDS Also many trees, but built **one after another**, where each new tree fixes the **mistakes** of the previous ones. This is **boosting**. **XGBoost** is a fast, powerful version — usually the top performer on tabular data like this.

EXAMPLE Tree 1 misclassifies certain churners; tree 2 focuses on those hard cases; and so on. The result is a strong combined model.

Bagging (Random Forest) = parallel trees, reduces variance. Boosting (XGBoost) = sequential trees, reduces bias. Know this contrast — it's a classic question.

SAY IT IN THE INTERVIEW "XGBoost is gradient boosting — trees added sequentially, each correcting the previous ones' errors. It usually wins on tabular data, so it's my main model."

15 Bias–variance tradeoff

IN SIMPLE WORDS **Bias** = model too simple, misses patterns (**underfitting**). **Variance** = model too complex, memorises noise (**overfitting**). We want the sweet spot in between.

EXAMPLE Logistic Regression → higher bias (simple). A deep single tree → high variance. Random Forest/XGBoost with tuning → balanced.

SAY IT IN THE INTERVIEW "High bias underfits, high variance overfits. I balance them with the right model complexity, regularization, and cross-validation."

16 Overfitting, underfitting & regularization (L1/L2)

IN SIMPLE WORDS **Overfitting**: great on training, poor on test (memorised, didn't learn). **Underfitting**: poor on both. **Regularization** penalises complexity to curb overfitting. **L1 (Lasso)** can shrink some weights to exactly zero (feature selection); **L2 (Ridge)** shrinks weights smoothly.

EXAMPLE Spotting overfitting: train accuracy 99%, test accuracy 74% — big gap. Add regularization or reduce tree depth.

SAY IT IN THE INTERVIEW "Overfitting is a big train-test gap. I fight it with regularization — L1 also zeroes out weak features, L2 just shrinks them — plus depth limits and early stopping."

17 Cross-validation

IN SIMPLE WORDS Instead of trusting one train/test split, **k-fold CV** splits data into k parts, trains on k-1 and tests on the last, rotating k times, then averages. A more reliable estimate of real performance.

EXAMPLE 5-fold CV gives 5 AUC scores; reporting their mean $\pm$ std shows the model is stable, not lucky on one split.

SAY IT IN THE INTERVIEW "I use k-fold cross-validation so my score isn't dependent on one lucky split — I report the mean and spread across folds."

18 Hyperparameter tuning

IN SIMPLE WORDS Hyperparameters are settings we choose *before* training (tree depth, learning rate, number of trees). **GridSearchCV** / **RandomizedSearchCV** try many combinations with cross-validation and pick the best.

EXAMPLE For XGBoost, tune `max_depth`, `learning_rate`, `n_estimators`. Random search is faster when the grid is large.

SAY IT IN THE INTERVIEW "I tune hyperparameters like depth and learning rate using cross-validated grid/random search, optimising for ROC-AUC rather than accuracy."

E · Measuring the Model

19 Confusion matrix

IN SIMPLE WORDS A 2×2 table of what the model got right and wrong. Everything else (precision, recall) comes from it.

	PREDICTED: CHURN	PREDICTED: STAY
ACTUAL: CHURN	True Positive (caught)	False Negative (missed churner)
ACTUAL: STAY	False Positive (false alarm)	True Negative

SAY IT IN THE INTERVIEW "The confusion matrix shows TP, FP, FN, TN. For churn, a False Negative — missing someone who leaves — is the costly error, so I watch it closely."

20 Precision, Recall & F1

IN SIMPLE WORDS **Precision** = of those we flagged as churn, how many really churned (controls wasted offers). **Recall** = of all real churners, how many we caught (controls missed customers). **F1** = balance of the two.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP}) \cdot \text{Recall} = \text{TP} / (\text{TP} + \text{FN}) \cdot \text{F1} = 2 \cdot \text{P} \cdot \text{R} / (\text{P} + \text{R})$$

EXAMPLE For churn we usually prioritise **recall** — missing a leaving customer costs more than sending one extra discount. If discounts were very expensive, we'd lean toward precision instead.

SAY IT IN THE INTERVIEW "Precision is how many of my churn flags were right; recall is how many real churners I caught. For churn I optimise recall, accepting some false alarms to avoid missing leavers."

21 ROC-AUC & PR-AUC

IN SIMPLE WORDS **ROC-AUC** measures how well the model *ranks* churners above non-churners across all thresholds — 0.5 is random, 1.0 is perfect. On imbalanced data, **PR-AUC** (precision-recall) is often more informative because it focuses on the rare positive class.

EXAMPLE AUC 0.85 means: pick a random churner and a random stayer — 85% of the time the model gives the churner a higher score.

SAY IT IN THE INTERVIEW "ROC-AUC shows ranking quality independent of threshold; for imbalanced churn I also report PR-AUC since it emphasises the minority churn class."

22 Threshold tuning

IN SIMPLE WORDS The model outputs a probability; we choose the cut-off that turns it into Yes/No. Default is 0.5, but we can move it. Lower threshold → catch more churners (higher recall) but more false alarms.

EXAMPLE Setting the threshold at 0.35 instead of 0.5 catches more at-risk customers — sensible when a retention offer is cheap relative to losing the customer.

SAY IT IN THE INTERVIEW "I don't blindly use 0.5 — I tune the decision threshold to the business cost of a miss versus a false alarm."

F · Explaining & Using It (the differentiator)

23 Feature importance & SHAP

IN SIMPLE WORDS **Feature importance** tells us which inputs mattered most overall. **SHAP** goes further and explains a **single prediction** — how much each feature pushed *this* customer toward churn. It turns a black box into something a business can trust.

EXAMPLE SHAP for one customer: "+0.30 month-to-month contract, +0.15 high monthly charge, -0.10 long tenure → final churn score 0.78." Now you can act on it.

SAY IT IN THE INTERVIEW "I use SHAP to explain individual predictions — it shows why a specific customer is flagged, which makes the model actionable and trustworthy for the business."

24 Customer value & risk ranking

IN SIMPLE WORDS Not every churner is worth the same. We combine **how likely** they are to leave with **how valuable** they are, and act on the top of that list first.

```
priority score = churn_probability × customer_value
```

EXAMPLE $\text{customer_value} \approx \text{MonthlyCharges} \times \text{expected remaining months}$. A ₹999 plan user at 60% risk outranks a ₹149 user at 90% risk for retention spend.

SAY IT IN THE INTERVIEW *"I rank customers by churn probability times their value, so retention budget goes where it saves the most revenue — not just to whoever is likeliest to leave."*

25 Uplift modeling & retention ROI

IN SIMPLE WORDS The smartest question isn't "who will churn?" but "who will **change their mind** if we give them an offer?" That's **uplift modeling**. Some customers stay anyway; some leave no matter what — target the *persuadables* in between, then estimate the money saved vs money spent (**ROI**).

EXAMPLE "Give the top 500 at-risk high-value customers a ₹50 offer → cost ₹25,000; expected retained revenue ₹4,00,000 → strong ROI." That one number turns a model into a business case — this is the project's differentiator.

SAY IT IN THE INTERVIEW *"Beyond predicting churn, I estimate the ROI of acting on it — ideally with uplift modeling to target customers whose behaviour an offer would actually change, not those who'd stay or leave regardless."*

One-line project pitch: "I built a telecom churn model — cleaned and balanced the data, compared Logistic Regression, Random Forest and XGBoost, evaluated on recall and ROC-AUC, explained predictions with SHAP, and added a retention-ROI layer that ranks which high-value customers to save. At Airtel scale, this is exactly how you'd reduce subscriber porting cost-effectively."